

FernUniversität in Hagen
Fakultät für Mathematik und Informatik
LG Kooperative Systeme

UDP Transport Options

Implementierung und Analyse von RFC 9868 in Rust

Masterarbeit
im Studiengang Master Informatik

vorgelegt von
Alan Bernstein
Matrikelnummer: 2068834

Erstprüfer: Prof. Dr. Christian Icking
Zweitprüferin: Dr. Lihong Ma

Hagen, 15.09.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	iv
Tabellenverzeichnis	v
Abkürzungsverzeichnis	vi
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum	2
1.2.2 Unidirektionalität	2
1.2.3 Herausforderungen	2
1.3 Zielsetzung	3
1.4 Abgrenzung und Umfang	4
1.5 Aufbau der Arbeit	4
1.6 KI-Einsatz	5
2 Technische und methodische Grundlagen	6
2.1 Internet Protocol (IP)	6
2.1.1 IPv4	6
2.2 Das User Datagram Protocol (UDP)	6
2.2.1 Protokollstruktur	6
2.2.2 Eigenschaften	8
2.3 Middleboxes und Protokoll-Ossifikation	9
2.3.1 Internet-Ossifikation	9
2.3.2 Auswirkungen auf Protokollerweiterungen	9
2.4 RFC 9868: Transport Options for UDP	9
2.4.1 Überblick	9
2.4.2 Die Surplus Area	9
2.4.3 Options-Struktur	10
2.4.4 Definierte Options-Typen	10
2.4.5 SAFE und UNSAFE Options	10
3 Methodik und KI-gestützte Entwicklung	11
3.1 Forschungsdesign	11
3.2 RFC-Auswertung und schrittweise Umsetzung	12
3.3 Beteiligte und ihre Rollen	13

3.4	Bekannte Schwächen der Sprachmodelle	14
3.5	Testarchitektur und Prüfprozess	15
3.6	Externe Referenzfälle	18
3.7	Reproduzierbarkeit, Provenienz und Grenzen	20
4	RFC-Analyse und Anforderungsmodell	22
4.1	Anforderungen	22
4.1.1	Funktionale Anforderungen	22
4.1.2	Nicht-funktionale Anforderungen	22
4.1.3	Speichereffizienz und Zero-Copy	22
4.2	Existierende Implementierungen	22
4.2.1	Linux Kernel	22
4.2.2	Userspace-Implementierungen	22
4.3	Herausforderungen	22
4.3.1	Zugriff auf die Surplus Area	22
4.3.2	NAT-Traversal	22
4.3.3	Interoperabilität	22
4.3.4	Alternative Implementierungsansätze	22
4.4	Technologieauswahl	23
4.4.1	Programmiersprache Rust	23
5	Entwurf und Implementierung	30
5.1	Architektur	30
5.1.1	Architekturüberblick	30
5.1.2	Projektstruktur	30
5.1.3	Datenmodell der UDP Options	30
5.2	TLV-Parser und Serializer	30
5.2.1	Parsing-Algorithmus	30
5.2.2	Serialisierung	30
5.3	Option Checksum (OCS)	30
5.3.1	Berechnung	30
5.3.2	Validierung	30
5.4	Raw-Socket-Zugriff und Plattform-Spezifika	30
5.4.1	Socket-Erstellung und Paketempfang	30
5.4.2	Plattform-Spezifika unter Linux	30
5.4.3	Build-Konfiguration	31
6	Evaluation und Diskussion	32
6.1	Test- und Messkonzept	32
6.2	Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden . .	32

6.3	Soll-Ist-Abgleich mit RFC 9868	32
7	Fazit und Ausblick	33
7.1	Beantwortung der Forschungsfragen	33
7.2	Ausblick	33
7.3	Schlusswort	33
	Literaturverzeichnis	34
A	Provenienzregister	38
	Eidesstattliche Erklärung	40

Abbildungsverzeichnis

3.1	Prüfprozess eines Pull Requests: lokale Prüfkette, CI-Pipeline und Freigabe	17
-----	---	----

Tabellenverzeichnis

3.1	Testarten der Entwicklung: Definition, Zweck und Einsatzzeitpunkt . . .	16
3.2	Externe Referenzfälle KI-gestützter Ergebnisse mit unabhängiger Prüfinstanz	19
4.1	Vergleich der betrachteten Systemprogrammiersprachen anhand der für die RFC 9868-Implementierung maßgeblichen Kriterien	28
A.1	Provenienzregister (Arbeitsstand 2026-08-13)	38

Abkürzungsverzeichnis

APC Alternate Payload Checksum.

AUTH Authentication Option.

DTLS Datagram Transport Layer Security.

EOL End of List.

FRAG Fragmentation Option.

MDS Maximum Datagram Size.

MRDS Maximum Reassembled Datagram Size.

NOP No Operation.

OCS Option Checksum.

PMTU Path Maximum Transmission Unit.

TIME Timestamp Option.

1. Einleitung

1.1 Motivation

Die ursprüngliche Spezifikation des User Datagram Protocol (UDP), RFC 768 aus dem Jahr 1980, umfasst 663 Wörter [1]. Die Erweiterung RFC 9868 wurde im Oktober 2025 veröffentlicht und führt unter dem Titel *Transport Options for UDP* erstmals einen allgemeinen Optionsmechanismus für UDP ein [2]. Zwischen beiden Dokumenten liegen 45 Jahre; die Klartextfassung des RFC 9868 ist mehr als 27-mal so umfangreich wie die des RFC 768.

Die Einfachheit von UDP ist dabei kein Mangel, sondern Entwurfsziel: Das Protokoll bietet lediglich Portnummern zur Demultiplexierung und eine Prüfsumme zur Fehlererkennung. Genau diese Einfachheit macht UDP zu einem der am häufigsten genutzten Transportprotokolle im Internet.

RFC 9868 greift Funktionen auf, die UDP ursprünglich der Anwendung oder einer darüberliegenden Schicht überließ: Transportfragmentierung, eine zusätzliche Nutzdatenprüfsumme sowie Optionen für Größen- und Zeitinformationen [2, Sec. 5, 11.3 und 11.8]. Für ihre Bereitstellung kommen grundsätzlich drei Wege in Betracht:

- Erweiterung innerhalb der Nutzdaten: Jede Anwendung definiert ein eigenes Rahmenformat in der UDP-Nutzlast.
- Ein neues Transportprotokoll: Alle gewünschten Mechanismen können so von Beginn an vorgesehen werden.
- Erweiterung des bestehenden Formats: UDP selbst erhält einen Optionsmechanismus, der für Bestandssysteme möglichst unsichtbar bleibt.

RFC 9868 wählt den dritten Weg (die Muster hinter diesen Wegen vergleicht Kapitel 2) und nutzt dafür die sogenannte *Surplus Area*: den möglichen Bereich zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms. Da das UDP-Length-Feld die Länge des UDP-Datagramms angibt und diese kleiner sein kann als die vom IP-Header implizierte Transportnutzlast, entsteht ein bisher ungenutzter Bereich für Erweiterungen. Auf dieser Grundlage definiert RFC 9868 ein erweiterbares Rahmenwerk für Transportoptionen.

Damit steht das minimalistische UDP vor seiner bislang größten Erweiterung. Nun stellt sich die Frage, die diese Arbeit als Leitfrage begleitet:

Wie lässt sich UDP um Transportoptionen erweitern, ohne seine grundlegenden Eigenschaften aufzugeben?

Die Arbeit untersucht diese Frage anhand einer Referenzimplementierung und auf ausgewählten realen Netzpfaden.

1.2 Problemstellung

Die Implementierung von UDP-Optionen gemäß RFC 9868 wirft drei Problemfelder auf, die diese Arbeit behandelt: die Beschaffenheit der Surplus Area selbst, die bewusste Unidirektionalität des Protokolls sowie die praktischen Implementierungsherausforderungen vom Zugriff aus dem Benutzerraum bis zum Verhalten von Zwischensystemen. Die folgenden Abschnitte behandeln sie in dieser Reihenfolge.

1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum

Anders als TCP besitzt der ursprüngliche UDP-Header kein explizites Optionsfeld. RFC 9868 platziert die Optionen deshalb außerhalb der durch das UDP-Length-Feld begrenzten Nutzdaten, aber noch innerhalb des IP-Datagramms. Damit bleibt der UDP-Header unverändert, die Implementierung muss jedoch zwei unterschiedliche Längengrenzen zuverlässig auswerten. Den genauen Aufbau der Surplus Area behandelt Abschnitt 2.4.2.

1.2.2 Unidirektionalität

RFC 9868 stellt explizit klar: „*UDP is unidirectional; UDP Options do not change that fact.*“ [2, Sec. 6].

Für Mechanismen, die Bidirektionalität voraussetzen, fordert RFC 9868 eine getrennte Spezifikation [2, Sec. 6]. RFC 9869 definiert einen solchen Einsatz von REQ und RES für die Pfad-MTU-Ermittlung [3, Sec. 3]. Diese Vorgabe schließt einen Datenaustausch in beiden Richtungen nicht aus. Sie bedeutet, dass die UDP-Optionsschicht keine Antwort selbstständig erzeugt. Nicht das Protokoll, sondern die Anwendung oder eine in ihrem Auftrag arbeitende Bibliothek löst eine Antwort aus.

1.2.3 Herausforderungen

Die zentrale Herausforderung bei der Implementierung von UDP Optionen liegt im Zugriff auf die Surplus Area. Existierende Betriebssystem-Stacks liefern über Standard-Socket-APIs ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten an die Anwendungsschicht; die Surplus Area wird stillschweigend verworfen

[2, Sec. 18]. Den Weg eines UDP-Pakets durch den Linux-Kernel bis zu dieser Socket-Schnittstelle dokumentieren Stephan und Wüstrich [4].

Eine weitere Herausforderung stellt die Kompatibilität mit Zwischensystemen dar. RFC 9868 berichtet Interoperabilitätstests, in denen Pakete mit Surplus Area Geräte zur Netzadressübersetzung (NAT, *Network Address Translation*) passierten, nennt aber auch ein Intrusion Detection System, das die Diskrepanz zwischen UDP-Length und IP-Length in der Standardeinstellung als Angriff meldet [2, Sec. 18]. Ob reale Netzpfade solche Pakete tatsächlich durchlassen, ist eine empirische Frage dieser Arbeit (FF2).

1.3 Zielsetzung

Die Problemstellung verdichtet sich zu zwei getrennten Hürden: Ein UDP-Options-Paket muss normgerecht erzeugt und empfangen werden, und seine Surplus Area muss die untersuchten Netzpfade unverändert überstehen.

Eine normgerechte Implementierung sagt nichts darüber aus, ob Zwischensysteme Datagramme mit Surplus Area unverändert passieren lassen. Ebenso nützt ein durchlässiger Netzpfad wenig, wenn Erzeugung und Auswertung der Optionen fehlerhaft sind. Aus dieser Trennung ergeben sich zwei Forschungsfragen:

- **FF1:** Welche Anforderungen des RFC 9868 lassen sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen?
- **FF2:** In welchem Umfang bleibt die *Surplus Area* auf den untersuchten realen Netzpfeiden bis zur Gegenstelle unverändert erhalten?

Das Ziel dieser Arbeit ist es, beide Fragen zu beantworten. Dazu entsteht zunächst eine an RFC 9868 ausgerichtete und systematisch auf Konformität geprüfte Bibliothek für UDP-Transportoptionen in Rust. Die auf Linux und IPv4 begrenzte Implementierung arbeitet im Benutzerraum mit Raw Sockets. Sie umfasst Parser und Serialisierer für UDP-Optionen nach dem TLV-Schema (*Type-Length-Value*), die Validierung der Optionsprüfsumme (*Option Checksum*, OCS) gemäß Abschnitt 9 des RFC 9868 sowie die verpflichtend zu unterstützenden Optionen EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES [2, Sec. 10].

Die Verifikation erfolgt mehrstufig: Komponententests prüfen die einzelnen Bestandteile, Integrationstests den Loopback-Pfad, und für reine Regeln wird ergänzend eine Lean-Spezifikation gepflegt. Lean dient als formale Gegenprobe für Prüfsummenarithmetik sowie Längen- und Anordnungsregeln; das Verhalten von Raw Sockets,

Betriebssystemen und Zwischensystemen bleibt Gegenstand empirischer Prüfungen. Dieses Prüfprogramm schafft die Grundlage für die Beantwortung von FF1.

Für FF2 vergleicht die Evaluation gewöhnliche UDP-Datagramme mit gleich großen Datagrammen, die eine Surplus Area enthalten, auf ausgewählten realen Netzpfeilen. Sie unterscheidet zwischen unveränderter Zustellung, Entfernung der Surplus Area und Paketverlust. Die Schlussfolgerungen bleiben auf die beobachteten Pfade begrenzt.

1.4 Abgrenzung und Umfang

Diese Arbeit setzt einen bewussten Rahmen. Sie beschränkt sich auf eine Umsetzung des RFC 9868 im Linux-Benutzerraum für IPv4. Gegenstand sind das TLV-Rahmenwerk, die OCS sowie die verpflichtend zu unterstützenden Optionen. Die Beschränkung auf den Benutzerraum ist eine vorab gesetzte Rahmenbedingung der Aufgabenstellung. Abschnitt 4.4 begründet die dafür gewählte Umsetzung mit Raw Sockets gegenüber kernelnahen Verfahren.

Außerhalb des Umfangs liegt der REQ/RES-Anwendungsfall zur Pfad-MTU-Ermittlung, den RFC 9869 definiert [3]. Die Implementierung unterstützt nur die in RFC 9868 festgelegten Formate von REQ und RES. Ebenfalls nicht implementiert wird die TIME-Option. RFC 9868 beschreibt zwar ihr Format und den Anwendungszweck, die Schätzung der Paketumlaufzeit (*Round-Trip Time*, RTT), legt aber kein nutzbares Protokoll fest. Die Kennungen für AUTH, UCMP und UENC sind in RFC 9868 lediglich reserviert und werden daher ebenfalls nicht implementiert (AUTH [2, Sec. 11.9], UCMP [2, Sec. 12.1], UENC [2, Sec. 12.2]). Kernel-Module werden nicht entwickelt.

IPv6 wird weder implementiert noch evaluiert. Die hier untersuchten Mechanismen des RFC 9868 (*Surplus Area*, OCS, TLV-Optionen und FRAG) lassen sich vollständig über IPv4 zeigen. Die Implementierung und ihre Konformitätsprüfung beziehen sich daher auf IPv4.

1.5 Aufbau der Arbeit

Nach dieser Einleitung folgen sechs Kapitel, die drei Blöcke bilden:

- **Grundlagen und Methodik:** Kapitel 2 führt die technischen und methodischen Grundlagen ein: das Internet Protocol, UDP, Erweiterungsmuster von Transportprotokollen, RFC 9868 mit der *Surplus Area* sowie die Grundbegriffe der eingesetzten Prüfverfahren. Kapitel 3 legt die Forschungsmetho-

dik offen, bevor mit ihr gearbeitet wird: das Forschungsdesign, die RFC-Auswertungsmethode und der Einsatz von KI.

- **Anwendungsteil:** Kapitel 4 überführt RFC 9868 mit dieser Methode in ein prüfbares Anforderungsmodell und begründet die Technologieauswahl. Kapitel 5 führt Entwurf und Implementierung der Bibliothek komponentenweise zusammen. Kapitel 6 berichtet die Ergebnisse zu beiden Forschungsfragen sowie eine deskriptive Auswertung der KI-gestützten Arbeitsweise.
- **Schluss:** Kapitel 7 wiederholt die Forschungsfragen, beantwortet sie einzeln und zieht die Bilanz zur Leitfrage.

Diese lineare Kapitelfolge ist eine rekonstruktive Darstellungsstruktur und kein Abbild des tatsächlichen Arbeitsprozesses. Die Umsetzung verlief iterativ und agenten-gestützt, sodass Entwurf, Implementierung und Test einander fortlaufend bedingten.

1.6 KI-Einsatz

Diese Arbeit ist unter durchgehendem Einsatz künstlicher Intelligenz (KI) entstanden. Der Einsatz betrifft Text und Software. Auf der Textseite erstellten KI-Werkzeuge Entwürfe, überarbeiteten Formulierungen, übernahmen die LaTeX-Formatierung und bedienten die LaTeX-Werkzeugkette. Zudem prüften sie den Text wiederholt gegen den Primärtext des RFC 9868. Auf der Softwareseite unterstützten sie Planung, Implementierung, Tests und Verifikation der Bibliothek. Der Autor entschied über die Übernahme der Ergebnisse und verantwortet sie.

Zum Einsatz kommen zwei KI-Werkzeuge (harness), die ein Sprachmodell mit Dateizugriff und Kommandoausführung verbinden: Claude Code von Anthropic und Codex von OpenAI. Ihre Rollenverteilung, die bekannten Schwächen dieser Arbeitsweise und die Prüfkette vor der Übernahme eines Ergebnisses behandelt Kapitel 3.

2. Technische und methodische Grundlagen

2.1 Internet Protocol (IP)

2.1.1 IPv4

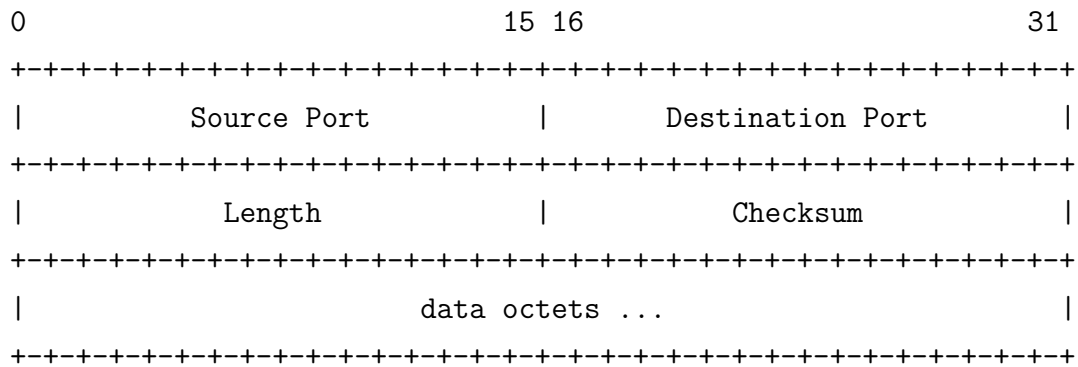
2.2 Das User Datagram Protocol (UDP)

Das User Datagram Protocol (UDP) ist neben dem Transmission Control Protocol (TCP) eines der beiden zentralen Transportprotokolle der TCP/IP-Protokollfamilie. Im Gegensatz zu TCP verfolgt UDP ein bewusst minimalistisches Design: Es sendet Datagramme ohne vorherigen Handshake oder Zustandsabgleich mit dem Empfänger, bietet keine Garantien für Zustellung, Reihenfolge oder Duplikaterkennung und verzichtet auf jeglichen Flusskontroll- oder Staukontrollmechanismus [1]. Mit der Protokollnummer 17 im IP-Header registriert, wurde UDP in RFC 768 spezifiziert und trägt die Bezeichnung STD 6 als Internet-Standard. Bemerkenswert ist, dass RFC 768 seit seiner Veröffentlichung im August 1980 über 45 Jahre hinweg keine formelle Aktualisierung erfahren hat. Erst RFC 9868 trägt den Header-Eintrag **Updates: 768** und erweitert damit den ursprünglichen UDP-Standard erstmalig direkt [2].

Historisch geht UDP auf die Aufspaltung des ursprünglichen *Transmission Control Program* zurück, das Cerf, Dalal und Sunshine Ende 1974 noch als monolithische Einheit aus Netzwerk- und Transportfunktionen beschrieben [5]. Mit deren Trennung in IP und TCP entstand der Bedarf nach einer schlanken, transaktionsorientierten Alternative ohne den Overhead von TCP, die Jon Postel 1980 spezifizierte [1]. Während der UDP-Kern über die folgenden Jahrzehnte unverändert blieb und nur Begleitdokumente Randfragen klärten, ließ erst die wachsende Bedeutung UDP-basierter Protokolle wie QUIC [6] und HTTP/3 [7] mit RFC 9868 die erste direkte Erweiterung des Standards entstehen.

2.2.1 Protokollstruktur

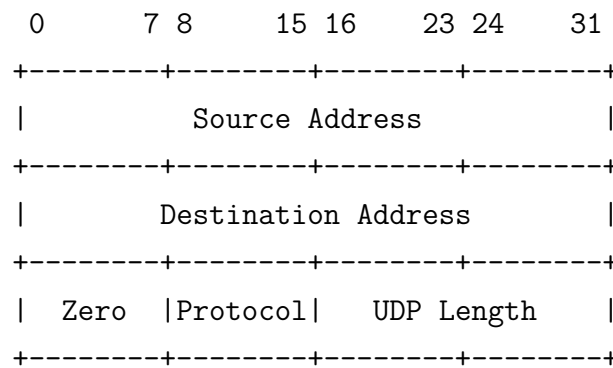
Ein UDP-Datagramm besteht aus einem Header mit fester Länge von 8 Byte und den darauf folgenden Nutzdaten. RFC 768 definiert den Header wie folgt [1]:



Die vier Felder des Headers haben jeweils eine Breite von 16 Bit:

- **Source Port:** Die Portnummer des sendenden Prozesses. Dieses Feld ist optional; wird es nicht benötigt, wird es auf null gesetzt [1]. In der Praxis nutzen Anwendungen den *Source Port* zur Identifikation des Absenders, damit der Empfänger Antworten an den korrekten Port zurücksenden kann.
- **Destination Port:** Die Portnummer des Zielprozesses auf dem empfangenden Host. Zusammen mit der IP-Zieladresse ermöglicht dieses Feld die *Demultiplexierung* eingehender Datagramme an die korrekte Anwendung.
- **Length:** Die Gesamtlänge des UDP-Datagramms in Byte, bestehend aus Header und Nutzdaten. Der Minimalwert beträgt 8, was einem Datagramm ohne Nutzdaten entspricht [1]. Da das Feld 16 Bit breit ist, ergibt sich eine theoretische Maximalgröße von 65 535 Byte.
- **Checksum:** Eine Prüfsumme über einen Pseudo-Header, den UDP-Header und die Nutzdaten. Die Berechnung verwendet das *Einerkomplement*-Verfahren: Alle 16-Bit-Wörter werden addiert und das Einerkomplement der Summe bildet den Prüfsummenwert [8]. Die Prüfsumme ist optional: wird sie nicht berechnet, wird das Feld auf null gesetzt [1].

Pseudo-Header. Die UDP-Prüfsumme schützt nicht nur die Transportschichtdaten, sondern bezieht über einen *Pseudo-Header* auch Informationen der Vermittlungsschicht ein. Dieses Konzept stellt sicher, dass ein Datagramm, das durch einen IP-Fehler an den falschen Host oder das falsche Protokoll zugestellt wird, erkannt und verworfen werden kann. RFC 768 definiert den folgenden Pseudo-Header [1]:



2.2.2 Eigenschaften

Die in Abschnitt 2.2.1 beschriebene Header-Struktur verdeutlicht bereits das minimalistische Design von UDP. Aus diesem Designziel leiten sich eine Reihe funktionaler Eigenschaften ab, die maßgeblich beeinflussen, für welche Anwendungsfälle das Protokoll geeignet ist.

Nachrichtenorientierung. UDP ist ein *nachrichtenorientiertes* Protokoll: Jede Sendeoperation erzeugt genau ein Datagramm, und der Empfänger erhält jede Nachricht als atomare Einheit. RFC 768 beschreibt UDP als Verfahren, mit dem Programme Nachrichten an andere Programme senden können („a procedure to send [...] messages to other programs“) [1].

Zustandslosigkeit. UDP verwaltet keinen protokollspezifischen Zustand für einzelne Kommunikationsbeziehungen und hält keinerlei Datenstrukturen wie Sequenznummern, Fenstergrößen oder Retransmission-Timer vor [2, Sec. 6].

Minimale Latenz. Da UDP verbindungslos arbeitet, entfällt jeglicher Handshake vor der Datenübertragung. Die erste Nachricht kann unmittelbar gesendet werden, ohne dass ein vorheriger Verbindungsaufbau abgewartet werden muss [9, Sec. 1]. Ebenso existiert kein Verbindungsabbau, der zusätzliche Latenz verursachen könnte.

Unabhängigkeit der Datagramme. Jedes UDP-Datagramm ist eine eigenständige, von allen anderen Datagrammen unabhängige Einheit. Der Verlust eines Datagramms hat daher keine Auswirkung auf die Verarbeitung nachfolgender Datagramme. Diese Eigenschaft vermeidet das sogenannte *Head-of-Line-Blocking* (HoL-Blocking), bei dem verlorene Pakete die Auslieferung nachfolgender Daten verzögern. Diese Eigenschaft von UDP war eine der Motivationen für die Entwicklung von QUIC: QUIC nutzt UDP als Transportschicht und realisiert auf Anwendungsebene ein Stream-Multiplexing, bei dem der Verlust eines Pakets nur den betroffenen Stream blockiert, nicht jedoch andere parallele Streams [6, Sec. 1].

Multicast und Broadcast. UDP ist verbindungslos und wird häufig als Transport für IP-Multicast und IP-Broadcast verwendet. Ein UDP-Datagramm kann an eine Multicast-Gruppe oder eine Broadcast-Adresse gesendet und von mehreren Empfängern empfangen werden, sofern Netzwerk und Hosts entsprechend konfiguriert sind. Dabei gelten die üblichen Eigenschaften von UDP: keine Garantie für Zustellung, Reihenfolge oder Duplikaterkennung. Diese Kombination eignet sich besonders für lokale Mechanismen wie Service Discovery sowie für bestimmte Echtzeitanwendungen in kontrollierten Netzen.

Diese Eigenschaften machen UDP zu einem attraktiven Transportprotokoll für latenzempfindliche, verlusttolerante oder gruppenbasierte Anwendungen. Allerdings fehlt UDP jede Form der protokolleigenen Erweiterbarkeit. Genau diese Lücke adressiert RFC 9868 durch die Nutzung der *Surplus Area* (siehe Abschnitt 2.4.2), um Optionen zu transportieren, ohne den bestehenden UDP-Header zu verändern.

2.3 Middleboxes und Protokoll-Ossifikation

2.3.1 Internet-Ossifikation

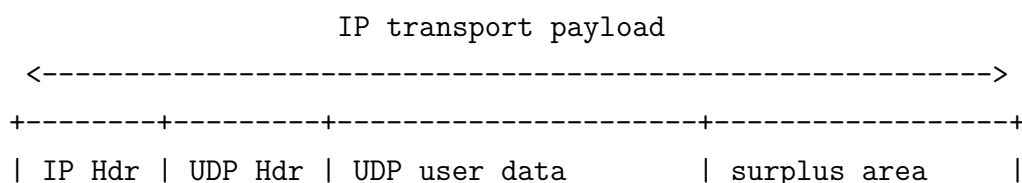
2.3.2 Auswirkungen auf Protokollerweiterungen

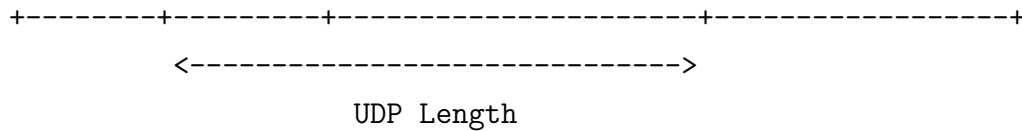
2.4 RFC 9868: Transport Options for UDP

2.4.1 Überblick

2.4.2 Die Surplus Area

Da das UDP-Length-Feld die Länge des UDP-Datagramms (Header und Nutzdaten) angibt und diese kleiner sein kann als die vom IP-Header implizierte Transportnutzlast, entsteht zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms ein bislang ungenutzter Bereich. RFC 9868 bezeichnet ihn als *Surplus Area* und verwendet ihn zur Aufnahme der UDP Options [2]:





Die *Surplus Area* beginnt, gegebenenfalls nach einem einzelnen Nullbyte zur Ausrichtung, mit der *Option Checksum (OCS)*, gefolgt von den eigentlichen Optionen im TLV-Format (Abschnitt 2.4.3). Diese Struktur ermöglicht variable Optionslängen und zukünftige Erweiterungen. Da Empfänger, die UDP Options nicht unterstützen, ausschließlich die durch das UDP-Length-Feld begrenzten Nutzdaten verarbeiten, bleibt die Erweiterung für Endsysteme typischerweise abwärtskompatibel; Ausnahmen bei Bestandssystemen und das Verhalten von Zwischensystemen behandeln [2, Sec. 18] beziehungsweise die Evaluation (Kapitel 6).

Wesentlich für die Einordnung ist, dass RFC 9868 kein neues Verhalten der Vermittlungsschicht einführt: Es definiert weder neue IP-Header-Felder noch verändert es die Verarbeitung von IPv4. Für die IP-Schicht sind die Bytes der *Surplus Area* reine Transportnutzlast innerhalb der durch IP **Total Length** begrenzten Datagramm-Größe; sie liegen lediglich hinter dem durch UDP **Length** markierten Ende der UDP-Nutzdaten. RFC 9868 ist damit ein reines Transport-Feature, das ausschließlich die zuvor ungenutzte Differenz zwischen UDP **Length** und der IP-Transportnutzlast belegt [2].

2.4.3 Options-Struktur

2.4.4 Definierte Options-Typen

2.4.5 SAFE und UNSAFE Options

3. Methodik und KI-gestützte Entwicklung

Diese Arbeit ist durchgehend mit Unterstützung großer Sprachmodelle (LLMs) entstanden: bei der Auswertung des RFC-Primärtexts, beim Entwurf und bei der Implementierung, bei der Analyse der Messkampagnen und bei der Prüfung von Zwischenergebnissen. Wissenschaftlich haltbar ist das nur, wenn die Arbeit offenlegt, wie sie Modellausgaben prüft und wer die Ergebnisse verantwortet. Diese Regeln müssen feststehen, bevor die Ergebnisse bewertet werden; aus diesem Grund steht dieses Kapitel vor der RFC-Analyse. Die KI-gestützte Entwicklung ist dabei Teil der Methode, keine eigene Forschungsfrage; Kapitel 6 wertet sie nur beschreibend aus. Das Kapitel beschreibt dazu zunächst das Forschungsdesign und die RFC-Auswertung, danach die Beteiligten mit ihren Rollen und die bekannten Schwächen der Sprachmodelle, anschließend die Testarchitektur der Entwicklung und externe Referenzfälle und abschließend Reproduzierbarkeit, Provenienz und Grenzen.

3.1 Forschungsdesign

Die Arbeit besteht aus drei Teilen, die aufeinander aufbauen und getrennt geprüft werden:

1. **Referenzimplementierung:** Es entsteht eine eigene Implementierung der UDP Transport Options in Rust, in dieser Arbeit *Referenzimplementierung* genannt.
2. **Konformitätsprüfung:** Es wird geprüft, ob diese Implementierung die Anforderungen von RFC 9868 erfüllt. Maßstab ist das Anforderungsmodell, das Kapitel 4 mit der Methode aus Abschnitt 3.2 aus dem Primärtext ableitet; das Ergebnis beantwortet Forschungsfrage FF1.
3. **Empirische Pfaduntersuchung:** Auf realen Netzpfaden wird gemessen, ob Pakete mit *Surplus Area* unverändert ankommen; das Ergebnis beantwortet FF2.

Dabei ist die Referenzimplementierung zugleich der Prüfgegenstand des zweiten und das Messinstrument des dritten Teils. Sprachmodelle sind in allen drei Teilen Werkzeug, aber in keinem die bewertende Instanz. Was gilt, entscheiden allein der RFC-Primärtext, Messdaten, ausführbare Prüfungen (Compiler, Tests, unabhängige Prüfprogramme) und der Autor. Jede Aussage eines Sprachmodells ist zunächst nur

ein Vorschlag: In die Arbeit geht sie erst ein, wenn eine dieser Instanzen sie bestätigt hat. In dem nächsten Abschnitt wird beschrieben, wie die Anforderungen aus dem RFC gewonnen und in Arbeitsschritte zerlegt werden.

3.2 RFC-Auswertung und schrittweise Umsetzung

Alle Anforderungen an die Implementierung stammen aus dem Primärtext von RFC 9868. Normative Schlüsselwörter wie **MUST**, **SHOULD** und **MAY** werden nach BCP 14¹ gelesen: Sie sind nur in Großschreibung normativ [10, 11]. Zusätzlich stellt RFC 9868 Absätzen mit solchen Schlüsselwörtern das Zeichenpaar >> voran. Dadurch sind die Anforderungen mechanisch auffindbar: Die eigene Zählung ergibt 103 solcher Absätze, und sie bilden das Gerüst der Anforderungsmatrix in Kapitel 4.

Dass solche Kontrollzahlen selbst fehleranfällig sind, zeigte sich früh: Eine erste Zählung ergab 96, weil sie tiefer eingerückte Markierungen in Listen übersah. Erst ein zweites, unabhängiges Zählverfahren deckte den Fehler auf. Aus diesem Grund gilt seither die Regel, Kontrollzahlen mit zwei unabhängigen Verfahren zu erheben.

Drei weitere Regeln sichern die Quellenarbeit:

1. Fundstellen werden satzgenau angegeben.
2. RFC 9868 und RFC 9869 bleiben strikt getrennt, denn RFC 9869 regelt die Pfad-MTU-Bestimmung (DPLPMTUD) für UDP-Optionen und liegt außerhalb dieser Arbeit [3].
3. Der Errata-Stand des RFC Editors wird einbezogen [12]; zum Stand 2026-08-13 sind das ein verifiziertes technisches Erratum zu Abschnitt 14 und ein redaktionelles zu Abschnitt 11.4.

[Offene Stelle: Beim Erreichen der Abschnitte 11.4 und 14 in der Anforderungsanalyse wird der dann aktuelle Errata-Stand erneut geprüft.]

Die gewonnenen Anforderungen werden nicht auf einmal umgesetzt, sondern in nummerierte Arbeitsschritte zerlegt. Die Schritte bauen aufeinander auf und sind wie folgt geordnet:

1. Prüfsummenmodul
2. Modell des Wire-Formats
3. TLV-Parser
4. Serialisierer

¹ BCP 14 fasst die beiden Regeldokumente RFC 2119 und RFC 8174 zusammen.

5. Options-Prüfsumme (OCS)
6. typisierte Optionen
7. Sende- und Empfangspfad
8. Fragmentierung mit Wiederzusammensetzung
9. öffentliche API

Jeder Schritt hat ein dokumentiertes Ziel und wird implementiert und geprüft, bevor der nächste beginnt; die Zerlegung liegt als Planverzeichnis im Implementierungs-Repository. Wer diese Schritte ausführt und wer sie prüft, regelt der nächste Abschnitt.

3.3 Beteiligte und ihre Rollen

Vier Arten von Akteuren wirken an der Arbeit mit: der Autor, zwei LLM-Werkzeuge, die Lean-Verifikation und die Prüfwerkzeuge. Dabei gelten zwei Regeln durchgehend: Kein Modell gibt Ergebnisse frei, und die Prüfung eines Entwurfs durch dasselbe Modell zählt nicht als unabhängige Prüfung.

Der *Autor* entscheidet, gewichtet, gibt frei und verantwortet das Ergebnis. Die beiden *LLM-Werkzeuge* sind Claude Code (Anthropic) als führendes Werkzeug und Codex (OpenAI) als Zweitprüfer; beide sind sogenannte *Harnesses*, also Programme, die ein Sprachmodell mit Dateizugriff, Kommandoausführung und dem Repository verbinden. Claude Code entwirft Code, Texte und Analysen. Codex dient vorrangig als unabhängiger Zweitprüfer und arbeitet in dieser Rolle nur lesend, mit dem ausdrücklichen Auftrag, Fehler und Gegenbeispiele zu finden. Änderungen durch Codex erfolgen ausschließlich in getrennten, vom Autor ausdrücklich freigegebenen Arbeitsaufträgen und zählen nicht als unabhängige Zweitprüfung. Für die Zweitprüfung ist bewusst ein Modell eines anderen Herstellers gewählt: Dadurch sollen sich die Fehlermuster der beiden Prüfer möglichst wenig überschneiden.

Die *Lean-Verifikation* liefert maschinengeprüfte Beweise, allerdings nur über handgeschriebene Modelle: Für ausgewählte Invarianten des Wire-Formats, der OCS, der Fragmentierung und der Empfangslogik existieren Lean-Modelle; ein Prüfskript baut ihre Beweise und kontrolliert, dass keine unzulässigen Axiome einfließen. Dennoch besteht eine wichtige Grenze: Nicht bewiesen ist, dass der Rust-Code diesen Modellen entspricht; die Kopplung besteht nur aus gespiegelten Konstanten und aus Tests. Somit ist die Aussage zulässig, dass ausgewählte Invarianten in Lean-Modellen bewiesen sind, nicht die Aussage, die Implementierung sei formal verifiziert.

Die *Prüfwerkzeuge* sind teils deterministisch (Compiler, Formatierungs- und Lint-Prüfung, Unit-Tests), teils arbeiten sie mit zufälligen Eingaben (Property- und Fuzz-Tests) oder von außen am fertigen Programm (Blackbox-Prüfung mit Paketmitschnitt). Abschnitt 3.5 definiert diese Testarten und ihre Einsatzzeitpunkte.

Im Umgang mit den Modellen gilt die Haltung, sie wie fähige, aber unerfahrene Entwickler zu behandeln, deren Arbeit grundsätzlich geprüft wird. Aus diesem Grund hat der Autor drei Arbeitsregeln schriftlich festgehalten:

1. Jedes Modellergebnis wird gegen den RFC-Primärtext geprüft, nie gegen eigene Zusammenfassungen.
2. Aufträge werden neutral formuliert, ohne das Modell in eine gewünschte Richtung zu lenken.
3. Widerspruch wird ausdrücklich ermutigt: Ein Modell, das „nein“ sagt oder Unsicherheit meldet, ist wertvoller als eines, das gefällig zustimmt.

Diese Regeln beschreiben den Sollprozess; welche Durchläufe tatsächlich belegt sind, verzeichnet das Provenienzregister (Abschnitt 3.7).

Aus diesen Rollen ergaben sich drei dokumentierte Arbeitsformen: die Entwicklung mit Pull-Request-Prüfung (Abschnitt 3.5), die abschnittsweise RFC-Lektüre, bei der vor jedem neuen Abschnitt grundsätzlich genau eine nur lesende Zweitmeinung des jeweils anderen Modells eingeholt wird, und die Messkampagnen, bei denen Erzeugung, Auswertung und Nachprüfung getrennt besetzt sind. Für frühere Projektphasen gilt eine Grenze: Arbeitsformen werden nur dort behauptet, wo sie protokolliert sind (Abschnitt 3.7). Warum diese Vorsicht durchgehend nötig ist, begründet der nächste Abschnitt.

3.4 Bekannte Schwächen der Sprachmodelle

Der Einsatz von Sprachmodellen bringt bekannte Schwächen mit, auf die die Methodik direkt antwortet. Drei davon betreffen diese Arbeit besonders:

Halluzination. Eine Halluzination ist ein überzeugend formulierter, aber falscher Inhalt. Kalai u. a. führen einen wesentlichen Teil dieser Fehler auf Trainings- und Bewertungsverfahren zurück, die sicheres Raten gegenüber eingestandener Unsicherheit belohnen [13].

Sycophancy (Gefälligkeit). Sycophancy bezeichnet die Neigung, sich der erkennbaren Erwartung des Gegenübers anzuschließen; sie ist bei größeren Modellen und nach Training mit menschlicher Rückmeldung verstärkt beobachtet worden [14, 15].

Beide Schwächen verstärken sich gegenseitig: Halluzination erzeugt falsche Aussagen, Sycophancy verhindert, dass sie auffallen.

Verfügbarkeit. Beide LLM-Werkzeuge sind Cloud-Dienste, die ausfallen, gedrosselt oder verändert werden können. Die Arbeit hängt deshalb an keiner Stelle von einem verfügbaren Dienst ab: Alle in die Arbeit übernommenen Ergebnisse liegen als Quelltext, Tests, Protokolle und Messdaten vor und sind ohne die Dienste prüfbar.

Die wichtigste Konsequenz folgt aus dem Nichtdeterminismus: Dieselbe Anfrage kann verschiedene Antworten liefern, und keine Antwort ist dadurch richtig, dass das Modell sie behauptet. Aus diesem Grund wird nie dem Modell geglaubt, ob die Implementierung die Norm erfüllt: Geprüft und belegt wird das mit der Testarchitektur aus Abschnitt 3.5 und der Konformitätsprüfung gegen den RFC; ein Beweis allgemeiner Korrektheit ist auch das nicht.

Dass diese Schwächen real auftreten, zeigen zwei Befunde aus dem Projekt. Eine Gegenprüfung der eigenen Sekundärdokumente gegen den RFC-Primärtext fand vier inhaltliche Abweichungen, unter anderem beim beschriebenen Berechnungsverfahren der OCS, während die Implementierung an den betroffenen Stellen konform war: Eigene Zusammenfassungen drifteten, deshalb wird immer am Primärtext geprüft. Und eine Kampagnennotiz nannte 15 Kontrollmessungen, tatsächlich waren es 18; drei Prüfinstanzen übersahen den Fehler zunächst. Keine Gegenmaßnahme ist eine Garantie: Was alle Prüfer teilen, etwa gleichgerichtete Fehlermuster zweier Modelle, bleibt als Restrisiko bestehen.

3.5 Testarchitektur und Prüfprozess

Damit stellt sich die Frage, wie eine Implementierung zu prüfen ist, deren Entwürfe von einem nicht deterministischen Werkzeug stammen. Die Antwort ist eine mehrstufige Testarchitektur: Sie läuft vollständig lokal vor jedem Pull Request, und einen Teil der Stufen wiederholt die kontinuierliche Integration (CI) auf GitHub. Tabelle 3.1 definiert die Testarten, ihren Zweck und ihren Einsatzzeitpunkt:

Tabelle 3.1: Testarten der Entwicklung: Definition, Zweck und Einsatzzeitpunkt

Testart	Definition und Zweck	Einsatzzeitpunkt
Unit-Tests	prüfen einzelne Funktionen gegen erwartete Werte, darunter drei handgerechnete OCS-Referenzvektoren	vor jedem Pull Request und in der CI
Property-Tests	erzeugen zufällige Eingaben und prüfen Invarianten, etwa dass Serialisieren und anschließendes Parsen die Eingabe unverändert zurückliefern; lokal mit 1024 Fällen je Eigenschaft	vor jedem Pull Request und in der CI
Fuzz-Tests	füttern neun Zielprogramme zeitbegrenzt mit zufälligen Bytefolgen und suchen Abstürze; gefundene Gegenbeispiele sind als Regressionstests eingefroren	vor jedem Pull Request; die Regressionstests bei jedem Testlauf
Integrations-tests	prüfen Sende- und Empfangsweg über echte Sockets, einschließlich eines Testlaufs mit Root-Rechten für Raw Sockets	vor jedem Pull Request
Blackbox-Prüfung	das fertige Testprogramm sendet festgelegte Szenarien auf einem Linux-Zielsystem; <code>tcpdump</code> zeichnet die Bytes nach dem Kernel auf, ein eigenständiges Python-Prüfprogramm dekodiert sie vollständig, <code>tshark</code> bestätigt die IPv4- und UDP-Felder	vor jedem Pull Request
Lean-Prüfung	baut die Beweise der Lean-Modelle und kontrolliert die verwendeten Axiome	vor jedem Pull Request und in der CI

Aus Tabelle 3.1 geht hervor, dass jede Testart vor jedem Pull Request läuft. Dabei ist zu beachten, dass die CI nur einen Teil der Stufen wiederholt: Die Blackbox-Prüfung und der Testlauf mit Root-Rechten² bleiben an das lokale Umfeld gebunden, weil sie ein Linux-Zielsystem beziehungsweise erhöhte Rechte benötigen.

Dabei richtet sich die Blackbox-Prüfung gegen eine besonders schwer erkennbare Fehlerklasse: Fehler, die Implementierung und Test teilen. Sender und Empfänger der Bibliothek nutzen denselben Serialisierungscode; ein beidseitig gleicher Fehler bestünde jeden Round-Trip-Test.

Aus diesem Grund wird hier ohne Code der Implementierung geprüft: `tcpdump` zeichnet die tatsächlich gesendeten Bytes auf, und das Python-Prüfprogramm dekodiert IPv4, UDP und die UDP-Optionen selbst und leitet alle Prüfsummen mit eigener Faltung nach RFC 1071 [8] und eigener CRC32C-Tabelle her; `tshark` be-

² Raw Sockets geben einem Programm direkten Zugriff auf IP-Pakete ohne die übliche Transportschicht des Betriebssystems; unter Linux erfordern sie deshalb erhöhte Rechte.

stätigt zusätzlich die IPv4- und UDP-Felder mit einem fremden Dissektor, einen Dissektor für UDP-Optionen besitzt es nicht. Ganz unabhängig ist auch das nicht: Die erwarteten Referenzbytes stammen aus der projekteigenen Formatdokumentation; die Unabhängigkeit gilt also für den Code, nicht für den Entwurf.

Aus demselben Grund existieren die OCS-Referenzvektoren: RFC 9868 enthält keinen Referenzwert für diese Prüfsumme, und ein beidseitig gleicher Rechenfehler wäre unsichtbar geblieben. Die drei handgerechneten, unabhängig nachgerechneten Vektoren sind bewusst nicht palindromisch, damit bytevertauschende Fehler auffallen. *[Offene Stelle: Eine externe Bestätigung dieser Vektoren steht aus; Wiedervorlage nach dem RFC-Prüfter zu Abschnitt 9.]*

Der Prüfprozess selbst hat zwei Stufen und wird in Abbildung 3.1 dargestellt: Lokal bündelt ein Skript alle Testarten der Tabelle als Kontrollliste vor jedem Pull Request. Auf GitHub durchläuft jeder Pull Request danach eine Pipeline aus drei Schritten: ein Rust-Prüfschritt (Formatierung, statische Analyse, Tests), ein Lean-Prüfschritt und erst nach deren Erfolg ein Schritt, der zwei getrennte Prüfanfragen stellt, an Claude Code zu Codequalität und Sicherheit und an Codex gezielt zur RFC-9868-Semantik.

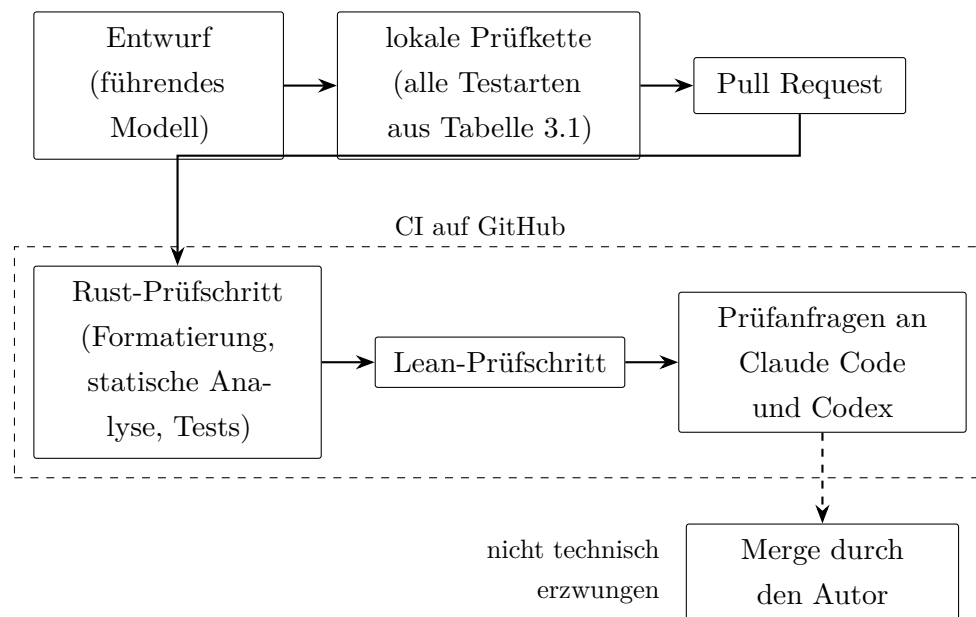


Abbildung 3.1: Prüfprozess eines Pull Requests: lokale Prüfkette, CI-Pipeline und Freigabe

Aus Abbildung 3.1 geht zugleich die wichtigste Grenze hervor: Die Prüfanfragen belegen nicht, dass beide Prüfungen auch abgeschlossen wurden, und kein Schritt ist als Merge-Bedingung technisch erzwungen; das Thesis-Repository hat gar keine solchen Prüfschritte. Dass diese Lücke real ist, zeigt Pull Request 27 des Implementierungs-Repositories: Der Merge erfolgte, bevor die Prüfungen abgeschlossen waren, und ein Befund des Zweitprüfers traf erst danach ein. Die Arbeit behauptet deshalb

kein technisch erzwungenes Vier-Augen-Prinzip; Aussagen über den Prüfprozess stützen sich auf Protokolle tatsächlich gelaufener Prüfungen.

Zwei Abgrenzungen schließen den Abschnitt ab. Erstens gehört diese Testarchitektur zur Entwicklung; die Pfaduntersuchung für FF2 nutzt eine eigene Messinfrastruktur (das fertige Binary auf entfernten Linux-Servern, Paketmitschnitte mit `tcpdump`, unabhängige Nachrechnung der Prüfsummen), die Kapitel 6 beschreibt. Ein bestandener Entwicklungstest sagt nichts über reale Netzpfade aus, und eine Messung ersetzt keinen Test. Zweitens setzt das Projekt bewusst kein Mutation-Testing am Quellcode ein; die Güte der Testsuite wird also nicht durch künstlich eingebaute Codefehler gemessen. Gezielt verfälschte Paketdaten kommen dagegen vor, etwa ein gekipptes Bit im Mitschnitt, mit dem die Blackbox-Prüfung ihre eigene Wachsamkeit belegt. Diese Grenze bleibt bestehen und wird im Ausblick wieder aufgegriffen.

3.6 Externe Referenzfälle

Das Muster dieser Methodik, dass ein generatives System vorschlägt, eine unabhängige Instanz prüft und Menschen freigeben, ist nicht projektspezifisch. Tabelle 3.2 ordnet acht publizierte Fälle KI-gestützter Ergebnisse danach, was die KI beitrug, wer unabhängig prüfte und welche Rolle Menschen behielten:

Tabelle 3.2: Externe Referenzfälle KI-gestützter Ergebnisse mit unabhängiger Prüfinstanz

Fall	KI-Beitrag	unabhängige Prüfinstanz	menschliche Rolle
AlphaFold [16]	Vorhersage von Proteinstrukturen	blinder CASP14-Vergleich mit experimentell bestimmten Strukturen	Bewertung, Einordnung
Davies u. a. [17]	Hypothesen und Kandidatenmuster für mathematische Zusammenhänge	Gegenbeispielsuche; klassischer Beweis	Beweis durch Mathematiker
FunSearch [18]	Programmkandidaten für offene Probleme	deterministischer Auswerter bewertet jede Ausgabe	Problemwahl, Analyse
AlphaProof [19]	Beweiskandidaten in formaler Sprache	der Lean-Kern verifiziert jeden Beweis maschinell	Aufgabenstellung, Bewertung
Knuths Zyklusproblem [20]	Konstruktion für ein offenes Zerlegungsproblem gerichteter Hamiltonkreise	Programmtests über viele Fälle; menschlicher Beweis; Lean-Formalisierung durch Dritte	Problemwahl, Beweis, Bericht
Zeta-Schranke (Anthropic) [21]	verbesserte Schranke zur Lage der Nullstellen der Riemannschen Zetafunktion	Lean-formalisierter Beweis; interne und externe Fachbegutachtung	Begutachtung, Veröffentlichung
Einheitsabstände (OpenAI) [22]	Gegenbeispiel zur Einheitsabstands-Vermutung von Erdős	Extraktion aus dem Modellprotokoll und mathematische Nachprüfung	Extraktion, Nachbeweis
C-Parameter-Resummation [23]	vollständige QCD-Rechnung, Numerik und Manuskript unter Physiker-Aufsicht	Validierung gegen Monte-Carlo-Simulationen; fachliche Aufsicht	Aufsicht, Verantwortung, Publikation

Aus der Tabelle wird ersichtlich, dass die Prüfinstanz je nach Fachgebiet wechselt, das Muster aber gleich bleibt: Die KI schlägt vor, geprüft wird woanders. Die vier Fälle aus dem Jahr 2026 benennen ihre Prüfinstanzen besonders klar. Bei Knuth stammte die Konstruktion vom Modell, der Beweis vom Menschen und die Lean-

Formalisierung von einem unabhängigen Dritten [20]. Die verbesserte Zeta-Schranke wurde als Lean-Beweis formalisiert und intern wie extern begutachtet [21].

Beim Gegenbeispiel zur Einheitsabstands-Vermutung extrahierten Mathematiker die Konstruktion aus dem Modellprotokoll und führten den Nachbeweis selbst [22]. Am weitesten geht der Physik-Preprint zur C-Parameter-Resummation: Sein Abstract legt offen, dass Rechnungen, Numerik und Manuskript von Claude unter Aufsicht eines Physikers erstellt wurden; geprüft wurde gegen Monte-Carlo-Simulationen [23]. Diese ausdrückliche Offenlegung in der Publikation selbst entspricht dem Prinzip, dem auch diese Arbeit folgt.

Als neunter Fall kommt die Migration der JavaScript-Laufzeit Bun von Zig nach Rust dem Zuschnitt dieser Arbeit am nächsten: agentengestützte Codeerzeugung mit getrennten Erzeuger- und Prüfrollen, allerdings alle aus derselben Modellfamilie besetzt [24, 25]. Trotz mehrstufiger Prüfung wurde nach dem Merge ein Fall undefinierten Verhaltens gemeldet; als Reaktion nahm das Projekt ein Prüfwerkzeug für undefiniertes Verhalten (Miri) in die kontinuierliche Integration auf [26].

Für diese Arbeit folgen daraus zwei Lehren: Die Zweitprüfung besetzt bewusst ein Modell eines anderen Herstellers, und auch eine mehrstufige Prüfung gilt nicht als Garantie. Alle neun Fälle teilen den Kern: Modellausgaben gelten erst nach unabhängiger Prüfung (Referenzdaten, mechanische Auswertung, formale Verifikation oder fachliche Begutachtung) und menschlicher Freigabe; keiner stützt sein Ergebnis allein auf die Zahl der eingesetzten Agenten.

3.7 Reproduzierbarkeit, Provenienz und Grenzen

Aussagen über den Entstehungsprozess sind nur so belastbar wie ihre Protokollierung. Die Arbeit führt deshalb ein Provenienzregister; *Provenienz* bezeichnet die dokumentierte Herkunft einer Aussage: wer oder was sie erzeugt hat, wer sie unabhängig geprüft hat und worauf sie sich stützt. Jede Zeile des Registers nennt Aussage oder Artefakt, Erzeugung, unabhängige Prüfung, Datum, Beleg und Status; die Werte „unbekannt“ und „nicht protokolliert“ sind zulässige, ehrliche Einträge. Die fortlaufend gefüllte Tabelle steht in Anhang A.

Für den Umgang damit gelten drei Grundsätze:

1. Eigene Zusammenfassungen und Arbeitsnotizen sind nur Suchhilfe, nie Beleg; geprüft wird am Repository, am Primärtext oder an den Messdaten selbst.
2. Pannen werden dokumentiert statt geglättet: Ein verlorener Prüfbericht wurde aus dem Sitzungsprotokoll wiederhergestellt und als Wiederherstellung gekennzeichnet, und eine versehentlich doppelt eingeholte Zweitprüfung wurde aufbewahrt und ausgewertet. Auch Knuth berichtet aus seinen Modellsitzungen,

dass Zwischenstände verloren gingen und das Modell an die Fortschrittsdokumentation erinnert werden musste [20].

3. Lücken werden benannt: Für frühe Projektphasen ist nicht protokolliert, welche Modelle und Konfigurationen im Einsatz waren; die betroffenen Registerzeilen tragen den Wert „nicht protokolliert“.

Damit sind auch die Grenzen der Methodik benannt. Die Prüfung mit zwei Modellen verschiedener Hersteller soll das Risiko gleichgerichteter Fehler verringern. Ob diese Wirkung eintritt, wird nicht gemessen; sie beweist weder die Unabhängigkeit der Prüfer noch die Fehlerfreiheit des Ergebnisses. Welche Befunde die Prüfungen im Projektverlauf tatsächlich lieferten und was sie übersahen, wertet Kapitel 6 beschreibend aus. Aussagen über die Wirksamkeit der KI-Kollaboration werden nicht getroffen.

Offene Stellen dieses Kapitels. Dieses Kapitel ist ein Arbeitsstand. Vier Punkte sind noch offen:

1. die Errata-Wiedervorlage zu den Abschnitten 11.4 und 14 (Abschnitt 3.2),
2. die externe Bestätigung der OCS-Referenzvektoren (Abschnitt 3.5),
3. die rückwirkende Füllung des Provenienzregisters in Anhang A,
4. die beschreibenden Kennzahlen der Kooperation, die erst nach Abschluss der Evaluation in Kapitel 6 berichtet werden.

4. RFC-Analyse und Anforderungsmodell

4.1 Anforderungen

4.1.1 Funktionale Anforderungen

4.1.2 Nicht-funktionale Anforderungen

4.1.3 Speichereffizienz und Zero-Copy

4.2 Existierende Implementierungen

4.2.1 Linux Kernel

4.2.2 Userspace-Implementierungen

4.3 Herausforderungen

4.3.1 Zugriff auf die Surplus Area

4.3.2 NAT-Traversal

4.3.3 Interoperabilität

4.3.4 Alternative Implementierungsansätze

4.4 Technologieauswahl

Die vorangegangene Analyse hat den Rahmen definiert, innerhalb dessen die Implementierung zu realisieren ist. Dieser Abschnitt überführt diese Analyse in die tragenden Technologie- und Methodenentscheidungen und begründet sie entlang der herausgearbeiteten Anforderungen.

4.4.1 Programmiersprache Rust

Die Wahl der Implementierungssprache wird nicht vorausgesetzt, sondern als Konsequenz der zuvor formulierten funktionalen und nicht-funktionalen Anforderungen entwickelt. Der Argumentationsbogen verläuft dabei in mehreren Schritten: von den Anforderungen, die das Vorhaben an eine geeignete Programmiersprache stellt, über die begriffliche Einordnung in die Klasse der Systemprogrammiersprachen und die Begründung der konkreten Sprachwahl bis zu einem Vergleich mit den Alternativen.

Aus dem RFC 9868 ergeben sich mehrere Anforderungen an die Programmiersprache, noch bevor eine konkrete Sprache benannt wird. Sie leiten sich unmittelbar aus den nicht-funktionalen Anforderungen ab. Zunächst wird ein direkter Zugriff auf *Raw Sockets* und damit auf rohe IP-Datagramme benötigt, da die Optionen von RFC 9868 in der *Surplus Area* jenseits der regulären UDP-Nutzdaten liegen und nur über selbst konstruierte Datagramme mit eigenem IP-Header (etwa mittels `IP_HDRINCL`) vollständig kontrolliert werden können. Hinzu kommt der Bedarf an einer bytengenauen Kontrolle in der *Surplus Area*, da die TLV-kodierten Optionen exakt an den durch den RFC vorgegebenen Bytegrenzen ausgerichtet sein müssen und jede implizite Umordnung oder Auffüllung durch die Sprache oder ihre Laufzeitumgebung die Korrektheit gefährden würde. Ebenso müssen potentiell bösartige eingehende Pakete ohne nennenswerten Laufzeit-Overhead verarbeitet werden, denn der Parser bildet die Vertrauensgrenze gegenüber dem Netz und darf weder durch präparierte Längfelder zum Absturz gebracht noch durch defensive Schutzmechanismen verlangsamt werden. Schließlich wird ein deterministisches Speicherverhalten ohne Garbage Collector vorausgesetzt, da solche Pausen das Laufzeitverhalten nicht-deterministisch machen würden. Diese Anforderungen bilden die Voraussetzungen zur Auswahl einer geeigneten Programmiersprache.

Sprachen, die diese Anforderungen erfüllen können, werden üblicherweise als *Systemprogrammiersprachen* zusammengefasst. Kennzeichnend für diese Klasse ist hardwarenahe Kontrolle über Speicher und Ausführung: direkter Zugriff auf Speicheradressen, Bitmuster und Betriebssystemschnittstellen ohne vermittelnde Abstraktionsschicht. Hinzu kommt eine explizite Speicherverwaltung ohne obligatorische

Laufzeitumgebung und insbesondere ohne Garbage Collector, sodass Allokation und Freigabe dem Programm und nicht einem im Hintergrund laufenden Prozess obliegen; der Quelltext wird zu nativem Maschinencode übersetzt, der ohne zwischengeschaltete virtuelle Maschine ausgeführt wird, woraus ein weitgehend deterministisches Laufzeitverhalten folgt. Eben diese Eigenschaften prädestinieren die Klasse für Aufgaben mit unmittelbarem Hardware- und Systembezug wie die Entwicklung von Betriebssystemen, Gerätetreibern und Netzwerkstacks. Genau in diese Schicht fällt die Implementierung eines Transportprotokoll-Mechanismus.

Die Definition wird von einer überschaubaren Gruppe etablierter Sprachen erfüllt, insbesondere von C, C++, Zig und Rust. Alle genannten Kandidaten bieten den geforderten hardwarenahen Zugriff und die Kompilation zu nativem Code; sie unterscheiden sich jedoch grundlegend darin, welche Sicherheitsgarantien sie dabei gewähren. Das entscheidende Kriterium ergibt sich daher nicht aus der Sprachklasse selbst, sondern aus der folgenden Anforderung: der sicheren Verarbeitung potentiell bösartiger Eingaben. Da der Parser die Vertrauensgrenze gegenüber dem Netz bildet, wird die standardmäßige Speichersicherheit zum entscheidenden Auswahlkriterium erhoben, denn eine Sprache, die Speicherfehler bereits konstruktiv ausschließt, verlagert eine ganze Klasse sicherheitskritischer Fehler von der Laufzeit in die Übersetzungsphase.

Dass diese Fehlerklasse kein akademisches Detail, sondern den weitaus größten Teil schwerwiegender Schwachstellen ausmacht, ist empirisch breit belegt: Sowohl bei Microsoft als auch im Chromium-Projekt werden übereinstimmend rund 70 Prozent der schwerwiegenden Sicherheitslücken auf Speichersicherheitsfehler zurückgeführt [27, 28], weshalb selbst die US-amerikanische National Security Agency (NSA) den Umstieg auf speichersichere Sprachen ausdrücklich empfiehlt und dabei Rust namentlich nennt [29]. Legt man die standardmäßige Speichersicherheit als Auswahlkriterium an das Feld an, so bleibt unter den etablierten Systemprogrammiersprachen Rust als einzige übrig, die diese Garantie ohne Garbage Collector und bereits zur Übersetzungsphase erbringt; die Wahl fällt folglich auf **Rust**.

Rust ist eine kompilierte, statisch und stark typisierte Systemprogrammiersprache, die ohne Garbage Collector und ohne obligatorische Laufzeitumgebung auskommt [30, 31]. Die statische Typisierung bedeutet dabei, dass die Typen aller Werte bereits zur Übersetzungszeit feststehen und vom Compiler geprüft werden, sodass eine ganze Klasse von Typfehlern vor der Ausführung und ohne Laufzeitkosten ausgeschlossen wird. Zugleich besteht über ein C-kompatibles *Foreign Function Interface* ein direkter Zugang zu den POSIX-Systemaufrufen, über die die benötigten *Raw Sockets* eingerichtet und die in der ersten Anforderung genannten IP-Datagramme kontrolliert

werden. Damit erfüllt Rust sämtliche Merkmale der zuvor entwickelten Definition: Es bietet hardwarenahe Kontrolle, verzichtet auf eine obligatorische Laufzeitumgebung und erzeugt deterministisch ausführbaren nativen Code.

Charakteristisch für Rust ist eine bewusste Auswahl der vorhandenen wie der absichtlich nicht vorhandenen Sprachmittel. Vorhanden sind das im nächsten Block ausführlich behandelte System aus *Ownership* und *Borrowing*; ferner algebraische Datentypen in Gestalt von `enum` mit vom Compiler auf Vollständigkeit geprüftem *Pattern Matching*; eine auf dem Typ `Result` aufbauende explizite Fehlerbehandlung; eine starke statische Typisierung; sowie das Schlüsselwort `unsafe` als bewusst markierte und eng begrenzte Brücke für jene Operationen, deren Sicherheit der Compiler nicht selbst nachweisen kann. Nicht vorhanden sind hingegen ein `null`-Wert, implizite Typkonvertierungen, klassische *Exceptions* sowie die Implementierungsvererbung objektorientierter Sprachen.

Gerade das Fehlen dieser Konstrukte verringert die Zahl der Fehlerquellen in einem sicherheitskritischen Parser erheblich. Diese Sprachmittel prägen den Aufbau der vorliegenden Implementierung unmittelbar. Sämtliche `unsafe`-Blöcke sind in der Socket-Schicht (`src/socket/`) konzentriert und dort hinter sichere Wrapper gekapselt, sodass der übrige Code ausschließlich mit sicheren Abstraktionen arbeitet.

Algebraische Datentypen und *Pattern Matching* bilden die TLV-kodierten Optionen der *Surplus Area* typsicher und erschöpfend ab, und die `Result`-basierte Fehlerbehandlung zwingt dazu, fehlerhafte oder bösartige Eingaben an jeder Verarbeitungsstelle explizit zu behandeln. Parser, Serializer und Prüfsummenberechnung sind dabei bewusst handgeschrieben und greifen nicht auf etablierte Bibliotheken wie `nom` oder `zerocopy` zurück, da die Mechanik von RFC 9868 selbst Gegenstand der Untersuchung ist und durch eine fertige Abstraktion nicht verdeckt werden soll.

Den Kern der Sprachwahl bildet die Art und Weise, in der Rust *Memory Safety* erzwingt: zum überwiegenden Teil bereits zur Übersetzungszeit durch das Typsystem, ergänzt um wenige, eng umrissene Laufzeitprüfungen. Unter *Memory Safety* wird die Abwesenheit einer Klasse von Speicherzugriffsfehlern verstanden, zu der insbesondere der *Buffer Overflow* (ein Zugriff jenseits der Grenzen eines reservierten Speicherbereichs), das *Use-after-free* (ein Zugriff auf bereits freigegebenen Speicher) und der *Data Race* (ein unsynchronisierter, nebenläufiger Zugriff mehrerer Ausführungsstränge, von denen mindestens einer schreibt) zählen. Rust begegnet diesen Fehlerklassen auf zwei Wegen: Das *Use-after-free* und der *Data Race* werden durch ein statisches Regelwerk ausgeschlossen, das aus drei aufeinander aufbauenden Konzepten besteht und das der Compiler vor der Codeerzeugung durchsetzt [31, 32]; der Zugriff jenseits der Grenzen eines Speicherbereichs hingegen wird, soweit er nicht ohnehin statisch

ausgeschlossen ist, durch automatisch eingefügte Bereichsprüfungen abgefangen, die bei einer Verletzung einen kontrollierten Programmabbruch (*panic*) auslösen, statt wie in C undefiniertes Verhalten zuzulassen [31, Kap. 9].

Das erste ist *Ownership*: Jeder Wert besitzt zu jedem Zeitpunkt genau einen Eigentümer; wird der Wert einer anderen Bindung zugewiesen oder an eine Funktion übergeben, so geht das Eigentum über (*move*), und die ursprüngliche Bindung ist anschließend nicht mehr verwendbar, was ein doppeltes Freigeben desselben Speichers ausschließt; verlässt der Eigentümer seinen Gültigkeitsbereich, wird der zugehörige Speicher deterministisch freigegeben, ohne dass es dafür eines Garbage Collectors bedürfte.

Das zweite ist *Borrowing*: Anstelle einer Eigentumsübertragung kann ein Wert über Referenzen geborgt werden, wobei zu jedem Zeitpunkt entweder beliebig viele geteilte, ausschließlich lesende Referenzen oder genau eine exklusive, schreibende Referenz bestehen dürfen, niemals jedoch beides zugleich; diese Regel schließt *Data Races* konstruktiv aus, da ein unsynchronisierter schreibender Zugriff bei gleichzeitig bestehenden weiteren Referenzen nicht typisierbar ist.

Das dritte sind *Lifetimes*: Sie binden die Gültigkeitsdauer jeder Referenz an jene des referenzierten Wertes und stellen so sicher, dass keine Referenz ihren Referenten überdauert, womit *Use-after-free* ausgeschlossen wird.

Die Gesamtheit dieser Regeln erzwingt der *Borrow Checker* vollständig zur Übersetzungszeit: Code, der gegen eine der Regeln verstößt, wird nicht übersetzt, und es entstehen keinerlei Laufzeitkosten für diese Prüfung. Das nachstehende Beispiel zeigt den Mechanismus an einem minimalen Fall, in dem eine geteilte und eine exklusive Referenz koexistieren sollen.

```
1 let mut buf = vec![1u8, 2, 3];
2 let view = &buf[0];           // geteilte (lesende) Referenz auf buf
3 buf.push(4);                  // Fehler: exklusiver Borrow, solange '
    view' lebt
4 println!("{view}");           // Nutzung von 'view' haelt den Borrow
    am Leben
```

Der Borrow Checker verweigert hier die Übersetzung mit dem Fehlercode E0502, weil `buf.push` eine exklusive Referenz auf den Puffer benötigt, während die geteilte Referenz `view` noch gebraucht wird; ein in C oder C++ an dieser Stelle möglicher Zugriff über einen durch die Reallokation ungültig gewordenen Zeiger ist damit bereits zur Übersetzungszeit ausgeschlossen:

```
1 --> src/main.rs:4:1
```

```

2 |
3 | let view = &buf[0];          // geteilte (lesende) Referenz auf
  | buf
4 |          --- immutable borrow occurs here
5 | buf.push(4);                // Fehler: exklusiver Borrow,
  | solange 'view' lebt
6 | ~~~~~ mutable borrow occurs here
7 | println!("{view}");         // Nutzung von 'view' haelt den
  | Borrow am Leben
8 |          ---- immutable borrow later used here
9 |
10 | For more information about this error, try 'rustc --explain E0502'.

```

Darauf aufbauend lässt sich die Wahl gegen die übrigen Kandidaten der Klasse präzise begründen, ohne deren jeweilige Stärken zu übergehen.

C bietet maximale, unverstellte Kontrolle über Speicher und Hardware und besitzt das mit Abstand reife Ökosystem für hardwarenahe Netzprogrammierung; diese Kontrolle wird jedoch ohne jede sprachseitige Sicherheitsgarantie gewährt, denn weder Schranken- noch Lebensdauerprüfungen für Zeiger sind Bestandteil der Sprachdefinition, und der Standard kennt eine große Zahl von Fällen mit *Undefined Behavior*, darunter gerade Pufferüberläufe und Zugriffe auf freigegebenen Speicher [33].

C++ erweitert dieses Fundament um mächtige Abstraktionsmittel und mildert mit *RAII* sowie Smart Pointern einen Teil der manuellen Speicherverwaltung, bleibt jedoch unsicher per Voreinstellung, da diese Werkzeuge optional sind und neben den unsicheren C-Altlasten fortbestehen, und erkaufte seine Ausdrucksstärke mit erheblicher Sprachkomplexität.

Zig ist eine moderne, bewusst einfach und explizit gehaltene Systemprogrammiersprache, die mit *comptime* eine ausdrucksstarke Auswertung zur Übersetzungszeit bietet und versteckte Allokationen vermeidet, indem Speicher ausschließlich über explizit übergebene Allocator-Objekte angefordert wird [34]; eine compilergarantierte Speichersicherheit nach Art des Borrow Checkers kennt Zig jedoch nicht, sondern erkennt nur eine Teilmenge unzulässiger Zugriffe durch Laufzeitprüfungen in den sicheren Build-Modi, die in den auf Geschwindigkeit oder Größe optimierten Modi entfallen, und hat mit Version 0.16.0 noch keine stabile Fassung erreicht [34]¹.

¹ Welche praktische Relevanz dieser Unterschied besitzt, zeigt die JavaScript-Laufzeit Bun, lange das größte Produktivprojekt in Zig: Nachdem allein in Version 1.3.14 vierzig Speicherlecks und zahlreiche Abstürze einzeln behoben worden waren, portierte das Team um Jarred Sumner den Kern von Zig nach Rust, um derartige Fehler strukturell auszuschließen, statt sie dauerhaft nachzubessern [24, 35]. Bun ist dabei kein Einzelfall: Auch der Paketmanager pnpm portiert seine Installations-Engine offiziell nach Rust; das zunächst separate Projekt paquet wird seit Juli 2026 im pnpm-Hauptrepositorium weitergeführt [36].

Rust nimmt in diesem Feld die Stellung ein, C-vergleichbare Kontrolle und Performance mit einer bereits zur Übersetzungszeit erzwungenen Speicher- und Data-Race-Freiheit zu verbinden; der Preis dafür ist die Einarbeitung in das Ownership-Modell, die jedoch durch präzise Compiler-Diagnosen unterstützt wird.

Abschließend stellt Tabelle 4.1 die entscheidungsrelevanten Eigenschaften der vier Kandidaten gegenüber.

Tabelle 4.1: Vergleich der betrachteten Systemprogrammiersprachen anhand der für die RFC 9868-Implementierung maßgeblichen Kriterien

Kriterium	C	C++	Zig	Rust
Speichersicherheit standardmäßig	nein	nein	nur in sicheren Build-Modi	✓ (Compiler)
Data-Race-Freiheit zur Übersetzungszeit	nein	nein	nein	✓
Speicherverwaltung	manuell	manuell, RAII	manuell, explizite Allocator	Ownership, kein GC
Laufzeit-Overhead	keiner, da ungeprüft	keiner, da ungeprüft	nur in sicheren Build-Modi	minimal (Be- reichsprüfung)
Compiler-Diagnostik als Rückkopplungs- schleife	begrenzt	begrenzt, oft unübersicht- lich	solide	ausgeprägt, mit stabilen Fehlercodes
Reife des Ökosystems (hardwarenahe Netze)	sehr hoch	sehr hoch	gering, noch nicht stabil	hoch, wachsend

Über die reinen Sicherheitsgarantien hinaus erweist sich der Rust-Compiler als eine starke, deterministische und maschinenlesbare Rückkopplungsschleife, was im Rahmen dieser Arbeit aus einem zweiten Grund bedeutsam ist. Seine Diagnosen benennen den Fehlerort nicht nur präzise, sondern tragen stabile Fehlercodes der Form `E0xxx` und ergänzen häufig konkrete, mit `help:` eingeleitete Korrekturvorschläge; ergänzt wird dies durch `cargo check` für eine schnelle Prüfung ohne vollständige Codegenerierung und durch den Linter `clippy` für eine weitergehende statische Analyse stilistischer und semantischer Schwachstellen [32].

Diese Eigenschaft gewinnt über den reinen Komfort beim Entwickeln hinaus an Bedeutung: Ein Compiler, der Fehler nicht nur meldet, sondern präzise lokalisiert und benennt, stellt ein hartes, automatisch und ohne menschliches Zutun prüfbares

Ziel bereit, gegen das sich jede einzelne Codeänderung verifizieren lässt, sodass fehlerhafte Konstruktionen nicht erst zur Laufzeit, sondern bereits bei der Übersetzung zurückgewiesen werden; genau das ist der Mechanismus, den maschinell erzeugter Quelltext benötigt. Wie sich diese Verbindung aus statischen Garantien und maschinenlesbarem Feedback methodisch nutzen lässt und in welchen Entwicklungsrahmen sie sich einfügt, behandelt Kapitel 3.

5. Entwurf und Implementierung

5.1 Architektur

5.1.1 Architekturüberblick

5.1.2 Projektstruktur

5.1.3 Datenmodell der UDP Options

5.2 TLV-Parser und Serializer

5.2.1 Parsing-Algorithmus

5.2.2 Serialisierung

5.3 Option Checksum (OCS)

5.3.1 Berechnung

5.3.2 Validierung

5.4 Raw-Socket-Zugriff und Plattform-Spezifika

5.4.1 Socket-Erstellung und Paketempfang

5.4.2 Plattform-Spezifika unter Linux

5.4.3 Build-Konfiguration

6. Evaluation und Diskussion

6.1 Test- und Messkonzept

6.2 Beobachtbarkeit und Middlebox- Verträglichkeit auf realen Pfaden

6.3 Soll-Ist-Abgleich mit RFC 9868

7. Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

7.2 Ausblick

7.3 Schlusswort

Literaturverzeichnis

- [1] J. Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, Aug. 1980. DOI: [10.17487/RFC0768](https://doi.org/10.17487/RFC0768). URL: <https://www.rfc-editor.org/info/rfc768>.
- [2] J. Touch. *Transport Options for UDP*. RFC 9868. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9868](https://doi.org/10.17487/RFC9868). URL: <https://www.rfc-editor.org/info/rfc9868>.
- [3] G. Fairhurst und T. Jones. *Datagram Packetization Layer Path MTU Discovery (DPLPMTUD) for UDP Options*. RFC 9869. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9869](https://doi.org/10.17487/RFC9869). URL: <https://www.rfc-editor.org/info/rfc9869>.
- [4] A. Stephan und L. Wüstrich. „The Path of a Packet Through the Linux Kernel“. In: *Seminar Innovative Internet Technologies and Mobile Communications (IITM), WS 23*. Technische Universität München, Chair of Network Architectures und Services, 2024, S. 89–93. DOI: [10.2313/NET-2024-04-1_16](https://doi.org/10.2313/NET-2024-04-1_16). URL: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2024-04-1/NET-2024-04-1_16.pdf (besucht am 13.08.2026).
- [5] V. Cerf, Y. Dalal und C. Sunshine. *Specification of Internet Transmission Control Program*. RFC 675. Internet Engineering Task Force, Dez. 1974. DOI: [10.17487/RFC0675](https://doi.org/10.17487/RFC0675). URL: <https://www.rfc-editor.org/info/rfc675>.
- [6] J. Iyengar und M. Thomson. *QUIC: A UDP-Based Multiplexed and Secure Transport*. RFC 9000. Internet Engineering Task Force, Mai 2021. DOI: [10.17487/RFC9000](https://doi.org/10.17487/RFC9000). URL: <https://www.rfc-editor.org/info/rfc9000>.
- [7] M. Bishop. *HTTP/3*. RFC 9114. Internet Engineering Task Force, Juni 2022. DOI: [10.17487/RFC9114](https://doi.org/10.17487/RFC9114). URL: <https://www.rfc-editor.org/info/rfc9114>.
- [8] R. Braden, D. Borman und C. Partridge. *Computing the Internet Checksum*. RFC 1071. Internet Engineering Task Force, Sep. 1988. DOI: [10.17487/RFC1071](https://doi.org/10.17487/RFC1071). URL: <https://www.rfc-editor.org/info/rfc1071>.
- [9] L. Eggert, G. Fairhurst und G. Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, März 2017. DOI: [10.17487/RFC8085](https://doi.org/10.17487/RFC8085). URL: <https://www.rfc-editor.org/info/rfc8085>.
- [10] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Internet Engineering Task Force, März 1997. DOI: [10.17487/RFC2119](https://doi.org/10.17487/RFC2119). URL: <https://www.rfc-editor.org/info/rfc2119>.

- [11] B. Leiba. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174. Internet Engineering Task Force, Mai 2017. DOI: [10.17487/RFC8174](https://doi.org/10.17487/RFC8174). URL: <https://www.rfc-editor.org/info/rfc8174>.
- [12] RFC Editor. *RFC Errata for RFC 9868*. RFC Editor. 2026. URL: <https://www.rfc-editor.org/errata/rfc9868> (besucht am 13.08.2026).
- [13] A. T. Kalai, O. Nachum, S. S. Vempala und E. Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664. 2025. URL: <https://arxiv.org/abs/2509.04664> (besucht am 13.08.2026).
- [14] E. Perez u. a. *Discovering Language Model Behaviors with Model-Written Evaluations*. arXiv:2212.09251. 2022. URL: <https://arxiv.org/abs/2212.09251> (besucht am 13.08.2026).
- [15] M. Cheng, C. Lee, P. Khadpe u. a. „Sycophantic AI decreases prosocial intentions and promotes dependence“. In: *Science* 391.6792 (2026). Artikelkennung eaec8352. DOI: [10.1126/science.aec8352](https://doi.org/10.1126/science.aec8352).
- [16] J. Jumper, R. Evans, A. Pritzel u. a. „Highly accurate protein structure prediction with AlphaFold“. In: *Nature* 596 (2021), S. 583–589. DOI: [10.1038/s41586-021-03819-2](https://doi.org/10.1038/s41586-021-03819-2).
- [17] A. Davies, P. Veličković, L. Buesing u. a. „Advancing mathematics by guiding human intuition with AI“. In: *Nature* 600 (2021), S. 70–74. DOI: [10.1038/s41586-021-04086-x](https://doi.org/10.1038/s41586-021-04086-x).
- [18] B. Romera-Paredes, M. Barekatin, A. Novikov u. a. „Mathematical discoveries from program search with large language models“. In: *Nature* 625 (2024), S. 468–475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- [19] T. Hubert, R. Mehta, L. Sartran u. a. „Olympiad-level formal mathematical reasoning with reinforcement learning“. In: *Nature* 651.8106 (2026). Online veröffentlicht am 12.11.2025, S. 607–613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- [20] D. E. Knuth. *Claude’s Cycles*. Vom 28.02.2026, überarbeitet am 14.04.2026. 2026. URL: <https://cs.stanford.edu/~knuth/papers/claude-cycles.pdf> (besucht am 13.08.2026).
- [21] Anthropic. *Learning more about Claude’s mathematical capabilities*. Vom 10.08.2026. 2026. URL: <https://www.anthropic.com/research/riemann-zeta> (besucht am 13.08.2026).
- [22] OpenAI. *An OpenAI model has disproved a central conjecture in discrete geometry*. Vom 20.05.2026. 2026. URL: <https://openai.com/index/model-disproves-discrete-geometry-conjecture/> (besucht am 13.08.2026).

- [23] M. D. Schwartz. *Resummation of the C-Parameter Sudakov Shoulder Using Effective Field Theory*. arXiv:2601.02484, eingereicht am 05.01.2026. 2026. URL: <https://arxiv.org/abs/2601.02484> (besucht am 13.08.2026).
- [24] *Rewriting Bun in Rust*. Oven (Bun). 2026. URL: <https://bun.com/blog/bun-in-rust> (besucht am 13.08.2026).
- [25] *Rewrite Bun in Rust*. oven-sh/bun, Pull Request 30412. Gemergt am 14.05.2026; 1.009.257 hinzugefügte Zeilen. Metadaten per GitHub-API verifiziert. GitHub. 2026. URL: <https://github.com/oven-sh/bun/pull/30412> (besucht am 13.08.2026).
- [26] *PathString::slice dangling reference UB - add Miri to CI (oven-sh/bun, Issue 30719)*. Gemeldet am 14.05.2026, nach dem Merge des Rust-Ports. GitHub. 2026. URL: <https://github.com/oven-sh/bun/issues/30719> (besucht am 13.08.2026).
- [27] M. Miller. *Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape*. Vortrag, BlueHat IL 2019, Tel Aviv. Microsoft Security Response Center, BlueHat IL 2019. 7. Feb. 2019. URL: <https://www.youtube.com/watch?v=PjbGojjnBZQ> (besucht am 31.05.2026).
- [28] The Chromium Project. *Memory safety*. The Chromium Project. URL: <https://www.chromium.org/Home/chromium-security/memory-safety/> (besucht am 31.05.2026).
- [29] National Security Agency. *Software Memory Safety. Cybersecurity Information Sheet*. National Security Agency (NSA). 10. Nov. 2022. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF (besucht am 31.05.2026).
- [30] The Rust Project. *Rust Programming Language*. 2025. URL: <https://www.rust-lang.org/> (besucht am 16.02.2026).
- [31] S. Klabnik und C. Nichols. *The Rust Programming Language*. 2025. URL: <https://doc.rust-lang.org/book/> (besucht am 16.02.2026).
- [32] The Rust Project. *The Rust Reference*. 2025. URL: <https://doc.rust-lang.org/reference/> (besucht am 16.02.2026).
- [33] ISO/IEC JTC 1/SC 22/WG14. *Programming languages: C*. N2310. Frei verfügbarer Arbeitsentwurf zu ISO/IEC 9899:2018 (C17). ISO/IEC JTC 1/SC 22/WG14. 2018. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (besucht am 31.05.2026).

- [34] Zig Software Foundation. *Zig Language Reference*. Version 0.16.0. Zig Software Foundation. 2026. URL: <https://ziglang.org/documentation/master/> (besucht am 15.05.2026).
- [35] J. Sumner. *40 memory leaks and dozens of crashes fixed this way in bun v1.3.14*. X (vormals Twitter). 2026. URL: <https://x.com/jarredsumner/status/2055934488526619129> (besucht am 31.05.2026).
- [36] pnpm-Projekt. *pacquet*. Offizielle Rust-Portierung der Installations-Engine von pnpm; Repositorium archiviert am 21.07.2026, Weiterentwicklung im Hauptrepositorium von pnpm. GitHub. 2026. URL: <https://github.com/pnpm/pacquet> (besucht am 13.08.2026).

A. Provenienzregister

Dieses Register setzt das in Abschnitt 3.7 beschriebene Schema um. Es wird fortlaufend und rückwirkend gefüllt; Lücken tragen die Werte „unbekannt“ oder „nicht protokolliert“. Interne Belege (Freeze-Dossiers, Sitzungsprotokolle, Kampagnenverzeichnisse) liegen aus Datenschutzgründen unversioniert beim Autor; Archive in `thesis/evidence/` sind über ihre SHA-256-Prüfsummen identifizierbar.

Tabelle A.1: Provenienzregister (Arbeitsstand 2026-08-13)

Aussage / Artefakt	Erzeugung	unabh. Prüfung	Datum	Beleg	Status
Zählung: 103 normative >>-Absätze in RFC 9868	Werkzeug und Modell	unabhängige Nachzählung mit zweitem Verfahren; früherer Wert 96 verworfen	2026-08-13 (letzte Prüfung)	<code>literature/rfc9868.txt</code>	bestätigt
drei handgerechnete OCS-Referenzvektoren, nicht palindromisch	Autor und Modell	unabhängige Nachrechnung; externe Bestätigung offen	2026-08-13	Impl.-Repo, Commit <code>6fa4ca1</code>	offen
Kampagnenarchiv <code>external-campaign-20260810T200118Z.tar.zst</code>	Messwerkzeuge	SHA-256 gegen <code>.sha256</code> -Datei	2026-08-13 (Prüfung)	<code>thesis/evidence/</code>	bestätigt
Kampagnenarchiv <code>bidir-campaign-20260811.tar.zst</code>	Messwerkzeuge	SHA-256 gegen <code>.sha256</code> -Datei; Prüfsummen dreifach nachgerechnet	2026-08-13 (Prüfung)	<code>thesis/evidence/</code>	bestätigt
GitHub-Zustand beider Repositories (Rulesets, Workflows, kein verpflichtender Check)	Werkzeug (API-Abfrage)	zweite Abfrage an zwei Tagen	2026-08-12 und 2026-08-13	Freeze-Dossier (lokal, unversioniert)	bestätigt
Audit-Bericht des Zweitmodells zum Implementierungsstand	Zweitmodell	Wiederherstellung aus Sitzungssprotokoll nach Bedienfehler	2026-07-10 bis 2026-08-13	Impl.-Repo, <code>review.md</code> (unversioniert)	vorhanden

Aussage / Artefakt	Erzeugung	unabh. Prüfung	Datum	Beleg	Status
Kapitelstart-Entscheidung (Methodik zuerst)	Autor und führendes Modell	ein unabhängiges Claude- und zwei Codex-Gutachten, nur lesend	2026-08-13	Sitzungsprotokoll (lokal)	bestätigt
Kapitel-3-Entwurf (Methodik)	führendes Modell	adversariale Prüfung durch das Zweitmodell; Befunde eingearbeitet	2026-08-13	Sitzungsprotokoll (lokal)	bestätigt
Quellenprüfung der externen Referenzfälle (3.7)	Werkzeug und Modell	Autor-Hinweise deckte einen vom Abrufwerkzeug ausgelassenen Abstract-Satz auf; Erstbewertung korrigiert	2026-08-13	Sitzungsprotokoll (lokal)	bestätigt
Kampagnen-Kontrollzahl in Abschnitt 3.4 (Notiz 15, belegt 18)	führendes Modell	Zweitmodell-Befund; eigene Nachprüfung am Kampagnenbericht (Erratum E1)	2026-08-13	Impl.-Repo, Kampagnenbericht	bestätigt
Modell- und Konfigurationsprovenienz früher Projektphasen	unbekannt	entfällt	nicht protokolliert	entfällt	offen

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder aus anderweitigen fremden Quellen entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

.....
Ort, Datum

.....
Unterschrift